LAB 5: SUPERVISED LEARNING

OBJECTIVES: Train models using supervised learning with labeled data.

THEORY:

Supervised learning involves training a model on a dataset containing input-output pairs (X, y) to learn a mapping function f(x) = y. The process involves minimizing a loss function that quantifies the error between predicted and actual labels. For binary classification, the model determines a decision boundary that separates classes in a high-dimensional feature space. Performance is quantified using metrics such as Accuracy (ratio of correct predictions), Precision (quality of positive predictions), Recall (ability to find all positive instances), and the F1-score (harmonic mean of precision and recall).

TOOLS: Scikit-learn, Keras

DATASET:

Breast Cancer Wisconsin Dataset: This dataset contains 569 instances with 30 numeric features representing characteristics of cell nuclei. The target labels are binary: Malignant (0) or Benign (1).

ALGORITHM

1. Start.
2. Load the Breast Cancer dataset and extract the feature matrix X and target vector y.
3. Apply standard scaling to X: X_scaled = (X - mean) / std_dev to ensure zero mean and unit variance.
4. Partition the data into training (X_train, y_train) and testing (X_test, y_test) sets.
5. Initialize a neural network with a sigmoid activation function in the output layer for binary probability mapping.
6. Compute the Binary Cross-Entropy loss: Loss = -1/N * sum(y * log(p) + (1-y) * log(1-p)).
7. Optimize weights using backpropagation to minimize the loss function over a set number of epochs.
8. Generate a prediction vector P where P_i = 1 if probability > 0.5, else 0.
9. Compute the Confusion Matrix entries: True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN).
10. Calculate Precision = TP / (TP + FP) and Recall = TP / (TP + FN).
11. End.

SOURCE CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Load and Scale Data
data = load_breast_cancer()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = Sequential([
    Dense(16, activation='relu', input_shape=(30,)),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid')])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

print("Training Classifier...")
history = model.fit(X_train, y_train, epochs=50, batch_size=10, verbose=0, validation_split=0.1)
y_pred_prob = model.predict(X_test)
y_pred = (y_pred_prob > 0.5).astype(int).flatten()

print("\n--- Performance Metrics ---")
print(classification_report(y_test, y_pred, target_names=data.target_names))
plt.figure(figsize=(10, 4))

# Subplot 1: Loss Curve
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Loss Convergence')
plt.xlabel('Epochs')
plt.legend()

# Subplot 2: Confusion Matrix
plt.subplot(1, 2, 2)
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=data.target_names,
yticklabels=data.target_names)
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()
```

OUTPUTS:

```
c:\Users\N I T R O\AppData\Local\Programs\Python\Python313\Lib\site-pack
  if not hasattr(np, "object"):
c:\Users\N I T R O\AppData\Local\Programs\Python\Python313\Lib\site-pack
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Training Classifier...
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step

--- Performance Metrics ---
              precision    recall  f1-score   support

   malignant       0.95      0.98      0.97        43
      benign       0.99      0.97      0.98        71

    accuracy                           0.97       114
   macro avg       0.97      0.97      0.97       114
weighted avg       0.97      0.97      0.97       114
```